Integrated Circuit Design
Final Project

Due: 2026/6/18 14:20

1 Problem Description

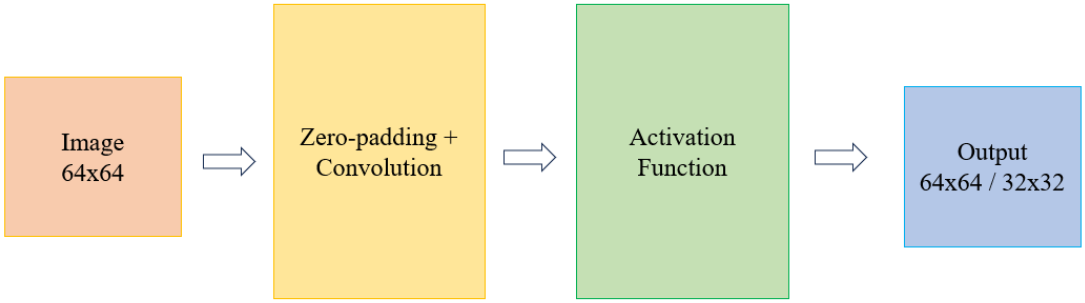

Figure 1: Architecture Overview.

Convolution is a fundamental technique in image processing and neural networks. By sliding a kernel (filter) across an image and performing multiply-and-accumulate (MAC) operations, convolution layers efficiently extract spatial features.

In this project, you are required to implement a hardware module named **core** that performs a convolutional operation on a 64×64 single-channel grayscale input image, followed by a nonlinear activation function. This module includes two processing stages executed sequentially:

Stage 1 – Zero-padding & Convolution

- **Input:** A 64×64 grayscale image is convolved with a 3×3 kernel. The input pixels are 8-bit unsigned and loaded in **raster-scan** order.
- **Kernel Weights:** Loaded as a 72-bit input, each weight is 8-bit signed fixed point number (2's complement).
- **Preprocessing:** Apply 1-pixel zero-padding before convolution.
- **Stride Modes:**
  - Stride-1: Produces a 64 × 64 output map.
  - Stride-2: Produces a 32 × 32 output map.

- **Arithmetic Precision:**
  - **Rounding:** The accumulation results should be rounded to the nearest integer (ties to positive infinity).

    - Do NOT truncate temporary results during the computations.

  - **Clamping:** After rounding, the final results should be clamped to **[0, 255]** before entering the next stage.

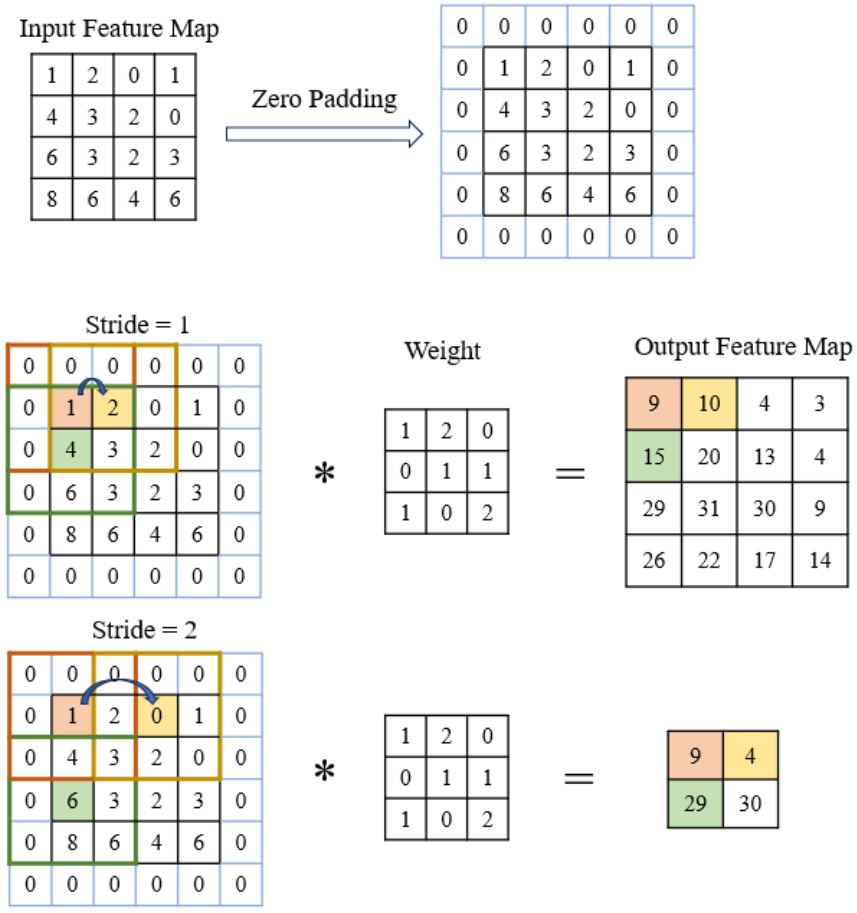

Figure 2: Zero Padding and Convolution (Stride = 1 or 2).

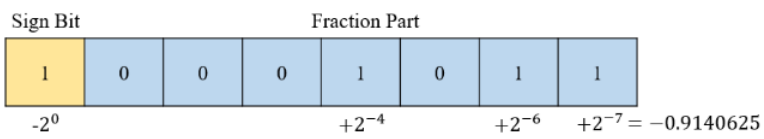

Figure 3: 8-bit Signed Fixed-Point Representation of Convolution Kernel Weights.

Stage 2 – Activation Function

- After the convolution operation, a nonlinear activation function $x^{2/3}$ is applied to the convolutional results.
- All clamped results in stage 1 should be **squared, then cube-rooted**.
  - Note: You **must** follow this order, or you might get the wrong results.
- The results of the cube root operation should be **truncated to an integer** and used as the final results.
- The final results should be written to the corresponding output memory in **raster-scan** order. The output address is determined by **o_out_addr**.

2 I/O Specifications

Signals	I/O	Bit Width	Description
i_clk	I	1	Clock signal
i_rst_n	I	1	Asynchronous negative-edge reset
i_in_valid	I	1	This signal is high if input **image** data is valid
i_in_data	I	32	Input image data (**4 pixels** each cycle)
i_stride_mode	I	1	0: stride=1, 1: stride=2
i_weight	I	72	Input kernel weight data
o_in_ready	O	1	Set to high if ready to get input **weight/image** data
o_out_data1	O	8	Output data
o_out_data2	O	8	
o_out_data3	O	8	
o_out_data4	O	8	
o_out_addr1	O	12	Address of output data
o_out_addr2	O	12	
o_out_addr3	O	12	
o_out_addr4	O	12	
o_out_valid1	O	1	Set to high with valid output data
o_out_valid2	O	1	
o_out_valid3	O	1	
o_out_valid4	O	1	
o_exe_finish	O	1	Set to high if the execution is finished

3 Waveform

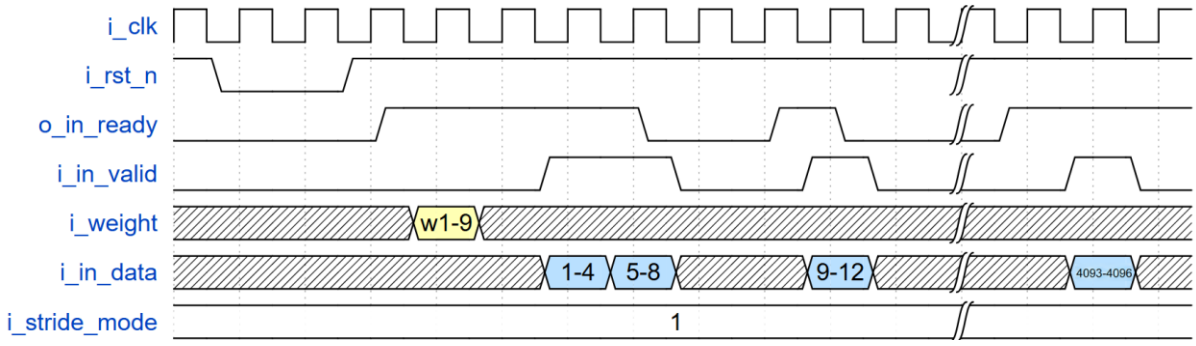

Figure 4: The Waveforms of I/O Signals (I).

- After **i_rst_n** is pulled high, you can assert **o_in_ready**. The kernel weights are then transferred via **i_weight** on the following negative edge and remain valid for **only one cycle**.

- A **one-cycle** gap is maintained after the weight transfer is complete. Subsequently, data transmission begins once **o_in_ready** is asserted.

- The input image pixels are loaded in raster-scan order through **i_in_data**, with 4 pixels each cycle. The input data is valid when **i_in_valid** is high. (Note: DON'T store all input pixels data in registers.)

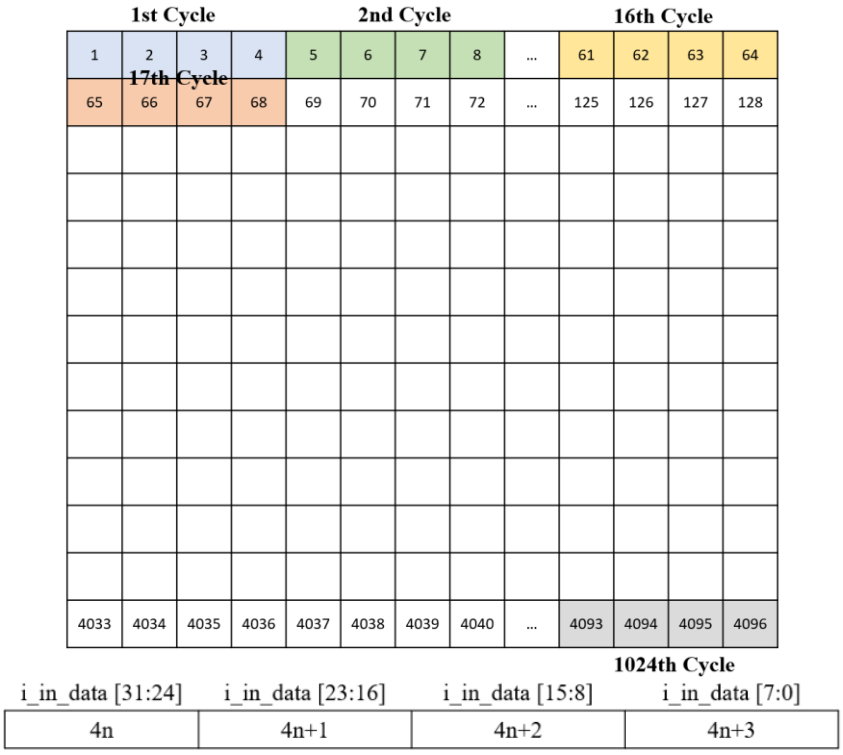

Figure 5: Input Data Order.

w1	w2	w3
w4	w5	w6
w7	w8	w9

i_weight[71:64]	i_weight[63:56]	i_weight[55:48]	i_weight[47:40]	i_weight[39:32]	i_weight[31:24]	i_weight[23:16]	i_weight[15:8]	i_weight[7:0]
w1	w2	w3	w4	w5	w6	w7	w8	w9

Figure 6: Input Weight Order.

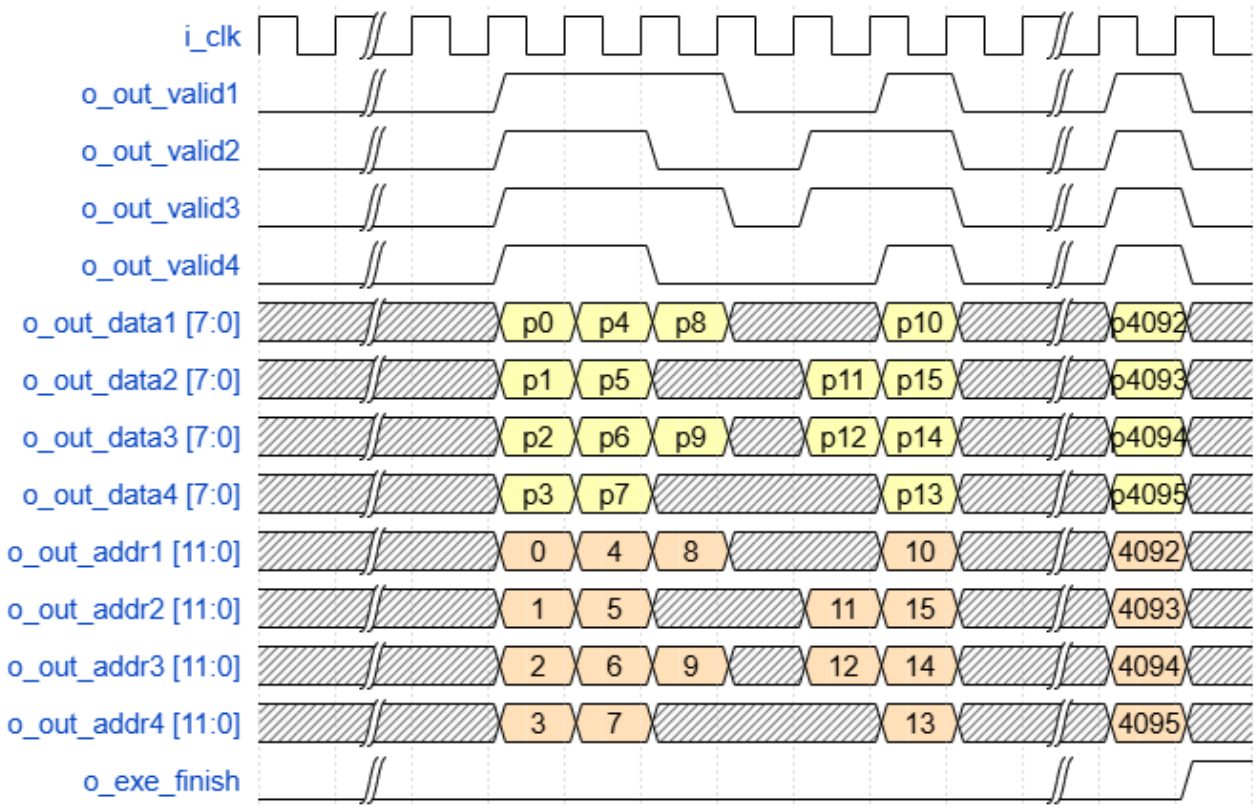

Figure 7: The Waveforms of I/O Signals (II).

- Pull **o_data_valid** high when the corresponding **o_data_out** is valid.
- The actual write order can be arbitrary as long as no two or more outputs are written to the same address at the same time.
- After all your results are outputted, you should pull **o_exe_finish** to high so the testbench will then check all your results.

4 Specification

- All input signals are synchronized with the negative edge clock.
- All outputs should be synchronized at clock rising edge.
- You should reset all your registers and outputs when **i_rst_n** is low.
- Your design should not exceed the maximum cycle of 1000000 for each pattern.

5 Rules

- Look-Up Table (LUT) is **strictly forbidden** in this project.
  - Do NOT use if-else or case to return cube roots without computation.
- Plagiarism in any form is **strictly prohibited**.
- TAs will use tools to check for plagiarism or the use of LUTs. If any violations or cheating (including hard-coding the outputs) are found, the team will receive a score of **zero**.

6 Files

Figure 8 shows the files you will be able to download from NTU cool.

- testbench.v: The testbench for this project.
- PATTERNS/: The patterns for simulation.
- core.v: The design file where you need to complete your RTL code.
- rtl.f: The files you need to use in RTL simulation.
- 01_run.sh: Command script for RTL simulation.
- core.sdc: The design constraints (Do NOT modify this file except for the cycle period).

- syn.tcl: Script for synthesis (Do NOT modify this file).
- 02_run.sh: Command script for synthesis.
- gate.f: The files you need to use in gate-level simulation.
- 03_GATE/cycle.txt: The clock period used for gate-level simulation (must be formatted to one decimal place).
- 03_run.sh: Command script for gate-level simulation.
- mmmc.view: The MMMC setting for APR.
- lab_script/: The scripts for APR.
- post.f: The files needed for post-layout simulation.
- 05_POST/cycle.txt: The clock period used for post-layout simulation (must be formatted to one decimal place).
- 05_run.sh: Command script for post-layout simulation.

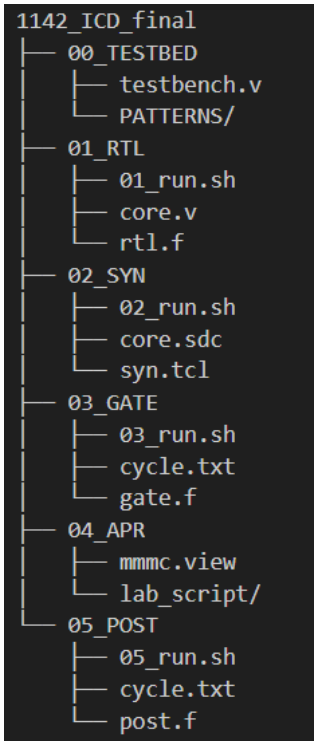

Figure 8: The Files You Will Get.

7 Simulation

- Use the following command to run RTL simulation in 01_RTL/:

  $ bash 01_run.sh

- Use the following command to run gate-level simulation in 03_GATE/:

  $ bash 03_run.sh

- Use the following command to run gate-level simulation in 05_POST/:

  $ bash 05_run.sh

- You can modify the testbench ID (tb1, tb2, tb3, tb4) in 01_run.sh, 03_run.sh, and 05_run.sh to test different patterns.

```
vcs -f rtl.f -full64 -R -debug_access+all -debug_region+cell +v2k +notimingcheck -sverilog +define+tb1 +define+CYCLE=10.0
```

Figure 9: 01_run.sh.

8 Synthesis

- Use the following command to run synthesis in 02_SYN/:

  $ bash 02_run.sh

- You can modify the clock cycle period in core.sdc.

```
set CLOCK "i_clk"
set cycle 10.0 ; # modify your clock cycle here
```

Figure 10: core.sdc.

- Do NOT modify any other content in core.sdc or the syn.tcl file.

- Make sure there are no errors, no latches, and no negative slack.

9 Place and Route

- Use the following command to open Innovus and follow the flow taught in class to finish the design in 04_APR/:

  $ innovus

- Ensure that all timing constraints are met.

10 Grading Policy

- (40%) Pass the RTL simulations. There are four public patterns, each worth 10 points.

- (10%) Pass gate-level simulations. Need to pass all public patterns to receive the credit. No timing violations or glitches after reset.

- (10%) Pass post-layout simulations. Need to pass all public patterns to receive the credit. No timing violations or glitches after reset.

- (10%) Report

  - Snapshot your APR result (refer to Figure 11).

    - Verify DRC result & Verify Regular Connectivity result.

    - Analyze Floorplan result.

    - **Core area** and the **simulation time**.

  - Specify your **cycle period**. Describe how you implement your design and the trade-offs between area and timing.

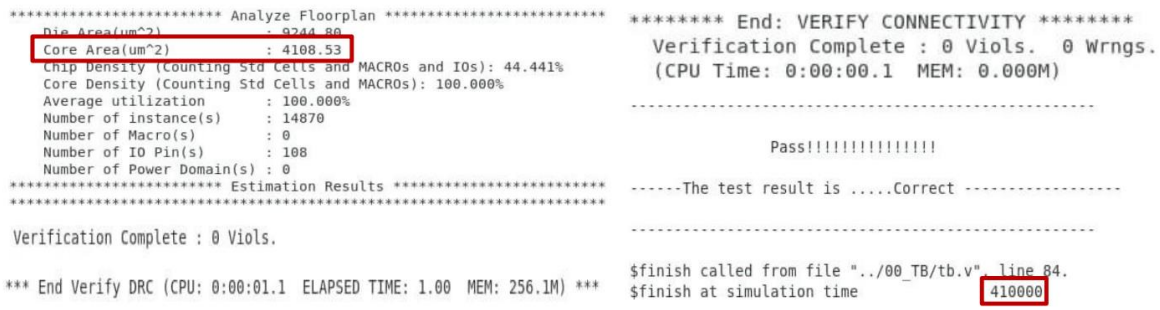

Figure 11: APR Result.

- (30%) Performance: If your team passes all public and hidden patterns in post-layout simulations, you will enter a contest, which is ranked by:

$$performance = Time \times Layout\ Area$$

The smaller value, the better. The best team will receive 30 points. For those who do not make it to the first contest but pass all public and hidden patterns in gate-level simulations, you will enter a second contest, which is similar to the first one but using the area reported by Design Compiler instead.

If there are k teams in the first contest, the top position for the team in the second test will be $k + 1$. So the j-th position in the second contest will have the final rank $j + k$.

The mapping of Final Rank (FR) to performance score:

FR 1: 30, FR 2: 28, FR 3: 26, FR 4: 24, FR 5: 22, FR 6: 20

FR $(6 + i)$: $(20 - i)$ for $i \geq 1$

11 Submission

- Upload **<student_id>_report.pdf** to NTU COOL.
- Save your APR design with the name "**final**".
  - This will generate **final** and **final.dat**, where **final.dat** is a directory.
- Create a folder named <student_id>_hw6/ , and follow the hierarchy in Figure 12 (next page).
- Copy **ic_design-hw6.tar.gz** to **share1/home/<your ADFP ID>/**
  - Compress your files with the instructions (example: b14901000)

    $ tar -zcvf ic_design-hw6.tar.gz b14901000_hw6/

  - Wrong file/folder format will get a penalty of 10 points.

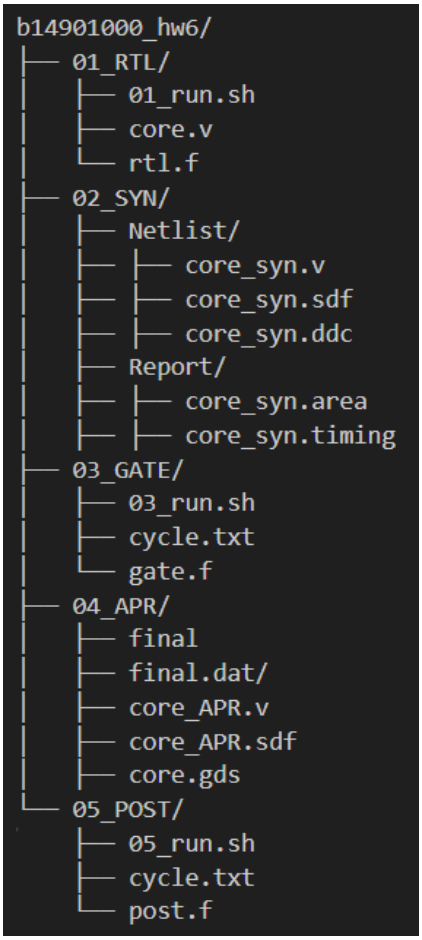

Figure 12: Submission Format.

TA

施伯儒 **r13943124@ntu.edu.tw**

林琮偉 **r13943010@ntu.edu.tw**